Clase 14 — Sockets: la API de la capa de transporte

Bloque: B (Aplicación) · **Semana:** 5 · **Duración estimada:** 120 min **Correlativa:** Clase 13 (correo) · **Sigue con:** Clase 15 (DHCP + Simulacro 2) **Material de referencia:** Tanenbaum cap. 6 · RFC 9293 · §14

🎯 Al terminar esta clase vas a poder

1. Explicar por qué la capa de transporte se programa con una **API** y no con "registros" del protocolo, y qué es un *socket*.
2. Escribir en C un **servidor TCP** y un **cliente TCP** completos y compilables, con el orden exacto de las llamadas.
3. Explicar para qué sirven / / y qué falla exactamente si se olvidan.
4. Diferenciar el modelo **con conexión** (TCP: + +) del **sin conexión** (UDP: /).
5. **Analizar críticamente** el pseudocódigo en C de una solución oficial de examen, encontrar sus errores y **escribir la versión corregida**.

> **Prerequisito del examen:** (30 puntos — con sockets UDP), (30 puntos — cliente anti-spam vía HTTP con consulta DNS por UDP). **Los dos problemas de 30 puntos de la capa de aplicación son de programación.**

PARTE 1 — TEORÍA

1.1 Qué es una API, y por qué la capa 4 se programa así

La capa de transporte es la **primera en que el sistema operativo implementa servicios para los procesos**. Antes (física, enlace, red) los datos viajan sin que ningún proceso los pida: un paquete de red es un paquete, va a donde diga la cabecera IP y a nadie le importa quién lo pidió.

A partir de TCP/UDP aparecen **las questions**: "¿este datagrama es para el navegador o para el correo?". Como **la CPU tiene un solo programa ejecutándose a la vez**, el SO necesita un mecanismo para:

- que un proceso **reserve un puerto** y así pedirle al mundo "lo que llegue por este número es mío";
- **entregar** cada segmento que llega al proceso correcto;
- **permitir** que un proceso espere o intente enviar sin bloquear la CPU entera.

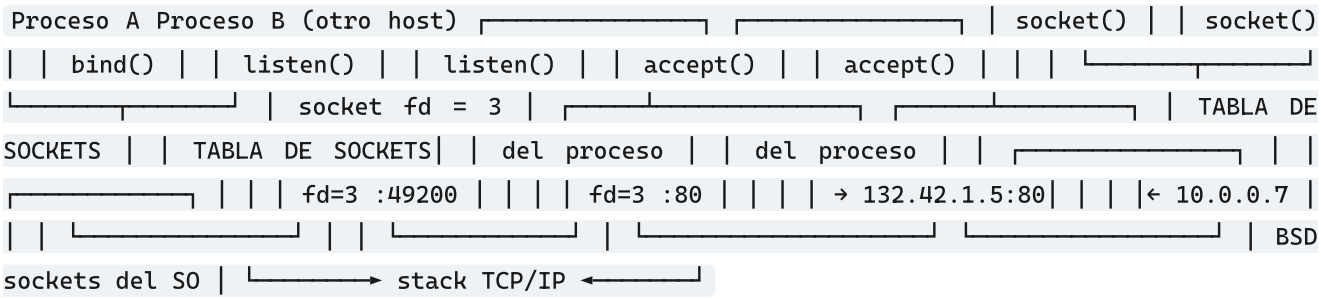
Ese mecanismo se llama **socket**. Y el conjunto de llamadas que el SO expone para usarlo se llama la **API de sockets**, que en Unix viene de **Berkeley (BSD)** y es el mismo en Linux, macOS y Windows (con diferencias de nombres).

> **Dato clave para el examen:** la API de sockets **nunca fue parte del protocolo TCP/IP**. Es una interfaz del sistema operativo. Por eso el mismo puede hablar TCP o UDP según qué creaste: **el protocolo lo decide el y el /, no el código de aplicación**.

Qué es un socket, en una frase

Un socket es **un par de descriptores de archivo más un estado**: el SO mantiene por cada socket las direcciones involucradas, el buffer de envío, el de recepción y la tabla de rutas interna (para TCP, el estado de la conexión completa).

En Unix, un socket es **un número entero** (el *file descriptor*), y por eso funciona con `read()`, `write()` y `close()` como cualquier archivo. Eso no es un detalle: es la razón de que la API sea tan simple.



1.2 La API de Berkeley/POSIX, llamada por llamada

Las que se usan siempre

Llamada	Qué hace	Disponible en
	Crea un socket y devuelve el file descriptor	TCP y UDP
	Fija la dirección local (IP y puerto)	ambos
	Marca el socket como **pasivo**: "acepto conexiones"	**solo TCP**
	Espera una conexión entrante y devuelve **un nuevo fd** para esa conexión	**solo TCP**
	Inicia la conexión hacia un destino	**solo TCP**
	Envía por una conexión **establecida**	ambos
	Recibe de una conexión establecida	ambos
	Envía **a un destino**, sin conexión previa	**solo UDP**
	Recibe **y quién lo mandó**	**solo UDP**
	Ajusta opciones del socket	ambos
	Libera el socket (cierra el fd)	ambos

El orden exacto: servidor TCP

Este es **el orden que se corrige en el examen**, y el orden importa porque cada llamada deja el socket en un estado que habilita la siguiente:

`1. socket(AF_INET, SOCK_STREAM, 0) | crear el socket. Devuelve fd = 3 (por ejemplo) ▼ 2. setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &uno, sizeof uno) | opcional pero recomendado: permite reabrir el puerto en el futuro | sin esperar al TIME_WAIT ▼ 3. bind(fd, (struct sockaddr*)&dir, sizeof dir) | FIJA IP y puerto LOCALES. OBLIGATORIO en el servidor: | sin esto el SO asigna un puerto efímero y nadie puede llegar ▼ 4. listen(fd, backlog) | pasa a estado PASIVO: el kernel empieza a ENCOLAR | conexiones entrantes en la cola de backlog ▼ 5. accept(fd, (struct sockaddr*)&cli, &len) | BLOQUEA hasta que llegue una conexión. | Devuelve un fd NUEVO, dedicado a esa conexión, | y llena cli con la IP y el puerto del cliente. | ¡OJO: el fd original NO se usa para atender la conexión! ▼ 6. send / recv sobre el fd NUEVO`

El error conceptual más común: creer que devuelve "la misma conexión" en el original. No: devuelve **un descriptor nuevo**. El original sigue siendo el "teléfono de la central" y se usa para el siguiente `accept()`.

El orden exacto: cliente TCP

`1. socket(AF_INET, SOCK_STREAM, 0) → fd 2. connect(fd, (struct sockaddr*)&destino, sizeof destino) | ¡NO HAY bind() NI listen()! | El SO elige un puerto efímero automáticamente | y hace el three-way handshake por debajo ▼ 3. send / recv sobre ese mismo fd`

El orden exacto: UDP, en ambos extremos

` SERVIDOR (desea conocer quién le habla)

1. socket(AF_INET, SOCK_DGRAM, 0) → SOCK_DGRAM, no SOCK_STREAM
2. bind(fd, (struct sockaddr*)&dir, sizeof dir) │ NECESARIO si querés un puerto fijo. │ Si no hacés bind, el SO asigna un puerto efímero │ y los clientes no pueden anticiparlo ▼
3. n = recvfrom(fd, buf, len, 0, (struct sockaddr*)&origen, &len_origen) │ ESPERA un datagrama. Además te dice QUIÉN lo mandó │ (IP y puerto), que es lo que TCP te da gratis ▼
4. sendto(fd, buf, n, 0, (struct sockaddr*)&origen, len_origen) │ respondés a la dirección que te devolvió recvfrom

CLIENTE 1. socket(AF_INET, SOCK_DGRAM, 0) 2. sendto(fd, buf, n, 0, (struct sockaddr*)&destino, sizeof destino) 3. recvfrom(fd, buf, len, 0, NULL, NULL) para escuchar respuestas `

> **Nota de examen:** y son a la vez "send/recv" **y** "connect/accept" combinados. No existen ni en UDP porque **no hay conexión que establecer**.

1.3 y el orden de bytes

La estructura

```
c struct sockaddr_in { sa_family_t sin_family; // AF_INET para IPv4 in_port_t sin_port; // puerto, EN ORDEN DE RED struct in_addr sin_addr; // IP, EN ORDEN DE RED // ... y un padding para que sea del tamaño de sockaddr };
```

El problema que resuelve

Las redes y las máquinas usan **ordenes de bytes distintos**:

- **big-endian** (*network byte order*): el byte más significativo **primero**. Es el estándar de todas las redes.
- **little-endian** (*host byte order*): el byte menos significativo primero. Es lo que usan x86, ARM en su modo normal, y prácticamente toda computadora de escritorio.

Si guardás el puerto 80 como número y lo mandás sin convertir:

` Puerto 80 en decimal = 0x0050

En little-endian en memoria: [50] [00] ← se manda primero el 0x50 Lo que recibe la red: 0x5000 = 20480 ← ¡puerto equivocado! En big-endian en memoria: [00] [50] ← correcto `

Por eso existen las conversoras:

Función	Qué hace
	host → network, **short** (16 bits, puertos)
	network → host, short
	host → network, **long** (32 bits, direcciones IP)
	network → host, long

> **Se usan igual en cliente y servidor.** No es "el servidor convierte y el cliente no". **Los dos lados** convierten, porque los dos escriben en un formato de red. Esto es una pregunta de examen clásica y casi todos la contestan mal.

y

`c // Forma clásica (devuelve INADDR_NONE si falla — no se puede distinguir error // de la dirección 255.255.255.255): struct in_addr a = inet_addr("132.42.120.5");

// Forma moderna y preferida (devuelve 1 si tuvo éxito, 0 si falló): struct in_addr a; inet_pton(AF_INET, "132.42.120.5", &a); `

1.4 Multiplexación y demultiplexación: el papel de

Aunque la teoría profunda está en la **Clase 16**, el punto de conexión con esta clase es uno solo y es el que más se pregunta:

` El SO mantiene una TABLA que asocia:

(protocolo, IP local, puerto local) → file descriptor

Cuando llega un datagrama UDP de la red: 1. el SO lee el puerto destino de la cabecera UDP 2. busca en la tabla (IP, puerto) 3. si está, ENTREGA los bytes al proceso dueño de ese fd 4. si no está, lo DESCARTA y a veces manda ICMP port unreachable `

es lo que **inscribe una fila en esa tabla**. Sin del lado servidor, no hay fila, y todo lo que llega se descarta.

El caso de TCP es un refinamiento: la tabla tiene — la **4-tupla** — porque dos clientes distintos pueden estar conectados al mismo puerto 80.

1.5 ⚠️ Actualización a 2026

Sigue igual — de memoria: - Las llamadas //////// /. **No cambió nada.** Los runtimes modernos (Go, Rust, Node, Python) las envuelven, pero el kernel sigue siendo BSD sockets. - La 4-tupla identifica una conexión TCP; la 2-tupla identifica un socket UDP **no conectado**. - / / , / , / . - El modelo cliente-servidor con /, y la programación multiproceso o multihilo.

Cambió: - **La herramienta de laboratorio.** Antes se usaba `tcpdump`; hoy lo estándar es **Wireshark** (libpcap/npcap, con filtros de display y **descodificación integrada** de QUIC, HTTP/3, TLS 1.3, DNSSEC y

DoH). sigue para captura masiva o headless. - **Ya no se puede sniffear HTTP/3 con captura pasiva.** El tráfico va cifrado en el cable con claves derivadas de TLS 1.3, y las cabeceras HTTP/3 viajan en QPACK. Para "ver" una sesión de HTTP/3 hace falta **instrumentación en el endpoint** (registro de QUIC con claves de depuración, o un proxy MITM). **Esto es un cambio real de la pedagogía del sniffing**, y explica por qué se enseña HTTPS y no HTTP/3 en el laboratorio. - **IPv6 dual-stack y direcciones v4-mapped.** es la forma IPv6 de escribir una dirección IPv4. Con (default en Linux), un socket acepta **también** tráfico IPv4. Es cómodo y es la razón de una larga lista de vulnerabilidades de clase y de bugs de validación. **Cualquier curso de sockets de hoy tiene que cubrirlo.** - **El matiz sobre la 2-tupla.** La clave real de demultiplexación en el kernel es la **5-tupla** . La "2-tupla (IP, puerto)" es la simplificación clásica para un socket UDP no conectado. Y lo que rompe el modelo de verdad es **QUIC**: multiplexa **muchas conexiones en el mismo 5-tupla** usando el **Connection ID** (§5.1) en lugar del 5-tupla.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Un servidor TCP de eco completo

Qué tiene que hacer: aceptar conexiones, leer lo que le mandan, devolvérselo, y seguir escuchando.

`c

include

include

include

include

include

include

include

define PUERTO 80

define MAX 1024

int main(void) { int fd, fd_cli, n; struct sockaddr_in dir, cli; socklen_t len; char buf[MAX];

/ *1. crear el socket: AF_INET (IPv4), SOCK_STREAM (TCP)* / fd = socket(AF_INET, SOCK_STREAM, 0); if (fd < 0) { perror("socket"); return 1; }

/ *2. (opcional pero recomendado) permitir reabrir el puerto enseguida* / int uno = 1; setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &uno, sizeof uno);

/ *3. fijar la direccion local* / memset(&dir, 0, sizeof dir); / *limpiar: sin esto sin_addr tiene basura* / dir.sin_family = AF_INET; dir.sin_port = htons(PUERTO); / *OJO: htons* / dir.sin_addr.s_addr = htonl(INADDR_ANY); / *cualquier interfaz* /

if (bind(fd, (struct sockaddr *)&dir, sizeof dir) < 0) { perror("bind"); return 1; }

/ *4. pasar a modo pasivo y encolar hasta 5 conexiones pendientes* / if (listen(fd, 5) < 0) { perror("listen"); return 1; }

printf("Escuchando en el puerto %d...\n", PUERTO);

/ *5. bucle: atender una conexion a la vez* / for (;;) { len = sizeof cli; fd_cli = accept(fd, (struct sockaddr *)&cli, &len); /* BLOQUEA */ if (fd_cli < 0) { perror("accept"); continue; }

/ *imprimir quien se conecto* / char ip[INET_ADDRSTRLEN]; inet_ntop(AF_INET, &cli.sin_addr, ip, sizeof ip); printf("Conexion desde %s:%d\n", ip, ntohs(cli.sin_port));

/ *6. atender mientras el cliente no cierre* / while ((n = recv(fd_cli, buf, MAX - 1, 0)) > 0) { buf[n] = '\0'; printf("Recibido (%d bytes): %s\n", n, buf); send(fd_cli, buf, n, 0); / *el eco* / }

close(fd_cli); / *el fd NUEVO se cierra, el original sigue* / printf("Conexion cerrada.\n"); } } `

Puntos para justificar en un examen:

- **antes** dear : si no, el padding tiene basura y puede fallar.
- y : **las dos conversiones**, siempre.
- devuelve **0** cuando el cliente cerró (FIN): ese es el **único** caso en que es falso sin error.
- devuelve **un fd nuevo**. El original nunca se usa para atender.
- = = "escuchar en todas las interfaces".

2.2 El cliente TCP

`c

include

include

include

include

include

include

int main(void) { int fd, n; struct sockaddr_in dest; char buf[1024] = "Hola servidor\n";

fd = socket(AF_INET, SOCK_STREAM, 0); if (fd < 0) { perror("socket"); return 1; }

memset(&dest, 0, sizeof dest); dest.sin_family = AF_INET; dest.sin_port = htons(80); / *OJO: htons* / inet_pton(AF_INET, "132.42.120.5", &dest.sin_addr);

/ *NO hay bind() ni listen() en el cliente* / if (connect(fd, (struct sockaddr *)&dest, sizeof dest) < 0) { perror("connect"); return 1; }

send(fd, buf, strlen(buf), 0);

n = recv(fd, buf, sizeof buf - 1, 0); if (n > 0) { buf[n] = '\0'; printf("Respuesta: %s", buf); }

close(fd); return 0; } `

2.3 Un par UDP que intercambia datagramas

`c / ---------- *SERVIDOR UDP* ---------- /

include

include

include

include

include

include

int main(void) { int fd, n; struct sockaddr_in dir, origen; socklen_t len; char buf[1024];

fd = socket(AF_INET, SOCK_DGRAM, 0); / *SOCK_DGRAM, no STREAM* / if (fd < 0) { perror("socket"); return 1; }

memset(&dir, 0, sizeof dir); dir.sin_family = AF_INET; dir.sin_port = htons(5000); dir.sin_addr.s_addr = htonl(INADDR_ANY);

if (bind(fd, (struct sockaddr *)&dir, sizeof dir) < 0) { perror("bind"); return 1; }

for (;;) { len = sizeof origen; n = recvfrom(fd, buf, sizeof buf - 1, 0, (struct sockaddr *)&origen, &len); if (n < 0) { perror("recvfrom"); break; } buf[n] = '\0';

char ip[INET_ADDRSTRLEN]; inet_ntop(AF_INET, &origen.sin_addr, ip, sizeof ip); printf("De %s:%d -> %s\n", ip, ntohs(origen.sin_port), buf);

/ *responder a QUIEN VINO, no a una direccion guardada aparte* / sendto(fd, "recibido\n", 9, 0, (struct sockaddr *)&origen, len); } close(fd); return 0; } `

`c / ---------- *CLIENTE UDP* ---------- / int main(void) { int fd; struct sockaddr_in dest; char buf[1024]; int n;

fd = socket(AF_INET, SOCK_DGRAM, 0);

memset(&dest, 0, sizeof dest); dest.sin_family = AF_INET; dest.sin_port = htons(5000); inet_pton(AF_INET, "10.0.0.7", &dest.sin_addr);

sendto(fd, "hola\n", 5, 0, (struct sockaddr *)&dest, sizeof dest);

n = recvfrom(fd, buf, sizeof buf - 1, 0, NULL, NULL); if (n > 0) { buf[n] = '\0'; printf("Respuesta: %s", buf); }

close(fd); return 0; } `

Las 4 diferencias con TCP, que es lo que se pregunta:

	TCP	UDP
Tipo de socket		
/	sí	**no existen**
/	/	**/**
¿Sabés quién envía?	sí, la conexión	**solo si usás **

2.4 Ejercicio 1 — ¿Por qué este servidor no funciona?

`c int main(void) { int fd, fd_cli; struct sockaddr_in dir; char buf[100];

fd = socket(AF_INET, SOCK_STREAM, 0);

memset(&dir, 0, sizeof dir); dir.sin_family = AF_INET; dir.sin_port = 80; / <-- *falta htons* /

bind(fd, (struct sockaddr *)&dir, sizeof dir); listen(fd, 5); fd_cli = accept(fd, NULL, NULL); /* <-- falta listen? no, esta bien */ recv(fd, buf, 100, 0); / <-- usa el fd equivocado */ send(fd, buf, 5, 0); / <-- idem */ return 0; } `

Resolución — los 3 errores:

1. ** `dir.sin_port = 80;` sin `htons`. **En little-endian, 80 = `0x0050` se escribe en memoria, y al enviarlo al cable el otro extremo lee `0x5000 = 20480`. El servidor queda escuchando en el puerto 20480.** El error clásico.**
2. ** y sobre el fd del listening socket.** Hay que usar `fd_cli`, el descriptor que devolvió `accept`. Con `fd` el comportamiento no es lo que uno espera, y lo que corresponde al socket pasivo no es el eco.
3. ** con en la dirección** — eso está permitido, pero si después querés responder por no vas a saber a quién. Con TCP usás sobre la conexión, así que es aceptable; no es error, es una decisión.

2.5 Ejercicio 2 — ¿Por qué el del servidor es obligatorio?

Pregunta: el servidor UDP de arriba hizo al puerto 5000. ¿Qué pasaría si lo sacáramos?

Resolución. Sin `bind()`, el SO le asigna un **puerto efímero** (típicamente entre 32768 y 60999 en Linux). El cliente no tiene forma de saber cuál es, porque:

- no está en ningún protocolo de descubrimiento;
- el cliente UDP **no recibe nada** hasta que envía primero;
- y no hay ningún mensaje de error que le diga "el servidor está en el puerto X".

O sea: **el servidor quedaría escuchando en un puerto que nadie conoce**, y el servicio sería inaccesible. En TCP esto se(compensa parcialmente) porque el cliente conoce el puerto **80** por convención; en UDP **no hay puerto conocido por convención**.

Por eso es **obligatorio en el servidor** y **opcional en el cliente**.

2.6 Ejercicio 3 — Dos procesos en el mismo puerto

Pregunta: ¿qué pasa si dos procesos hacen al mismo puerto TCP?

Resolución. Sin `SO_REUSEADDR`, el **segundo** `bind()` **falla** con ("address already in use"). Eso es lo correcto: dos procesos no pueden atender el mismo puerto, porque las conexiones llegarían a cualquiera de los dos al azar.

Con (en Linux también), **ambos** pueden hacer bind y el SO reparte las conexiones entrantes entre ellos según el hash de la 4-tupla. Es lo que usan los servidores nginx para tener N workers. Ojo: en TCP **no** permite que dos sockets **escuchen** al mismo tiempo y roben traffic de forma insegura; en UDP **sí** es peligroso, porque cada datagrama se lo lleva un proceso cualquiera.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Problema 1 (30 puntos) —

> **Fuente:** Examen / , 14 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf

Enunciado (texto del examen)

> Se desea implementar un sistema para brindar un servicio de conexión inalámbrica móvil a Internet, donde el usuario puede elegir el proveedor que le ofrezca un precio más bajo.
>
> Este sistema se implementará como una API en el equipo del usuario con la función y un servicio que se ejecuta en los puntos de acceso (AP).
>
> Para conectarse a Internet, la API del usuario deberá informarse del precio de cada punto de acceso a su alcance y luego conectarse al más barato. Si el punto de acceso más barato rechaza la conexión, se intentará con el siguiente, y así hasta lograr conectarse o no tener mas puntos de acceso disponibles.
>
> En tanto, el servicio del punto de acceso deberá comunicar su precio actual a los usuarios.
>
> Los puntos de acceso pueden aceptar un máximo de conectados.
>
> En los equipos de los usuarios se cuenta con las siguientes primitivas: - retorna la lista de APs dentro del alcance como una lista de SSID. - , se asocia al AP con el ssid dado. Los APs cuentan con un servicio DHCP por el cual en el momento de asociarse también se le asigna al usuario una dirección IP y direcciones de DNS. - , informa al sistema que el usuario se encuentra conectado a un AP y se pueden enviar datos.
>
> En los APs se cuenta con las siguientes primitivas: - , retorna la cantidad de usuarios conectados a este punto de acceso. - , devuelve el precio actual del punto de acceso.
>
> Se pide: 1. Especificar todos los mensajes necesarios para la comunicación. 2. Dibujar las máquinas de estado que especifiquen los procesos usuario y punto de acceso. 3. Implementar en un lenguaje de alto nivel las máquinas de estado definidas en el punto anterior. La comunicación entre usuarios y puntos de acceso se deberá realizar mediante sockets UDP. El usuario recibe las ofertas de precio en el puerto 12345.

Parte 1 — Los mensajes del protocolo

Solución oficial: , , , .

Análisis — por qué esos cuatro y no otros:

Mensaje	Quién lo manda	Para qué	Payload
****	AP → usuarios (broadcast)	"este soy yo y este es mi precio"	precio (int)
****	usuario → AP	"quiero conectarme a vos"	nada (o id de usuario)
****	AP → usuario	"aceptado, ya estás conectado"	id de sesión
****	AP → usuario	"no, tengo "	motivo

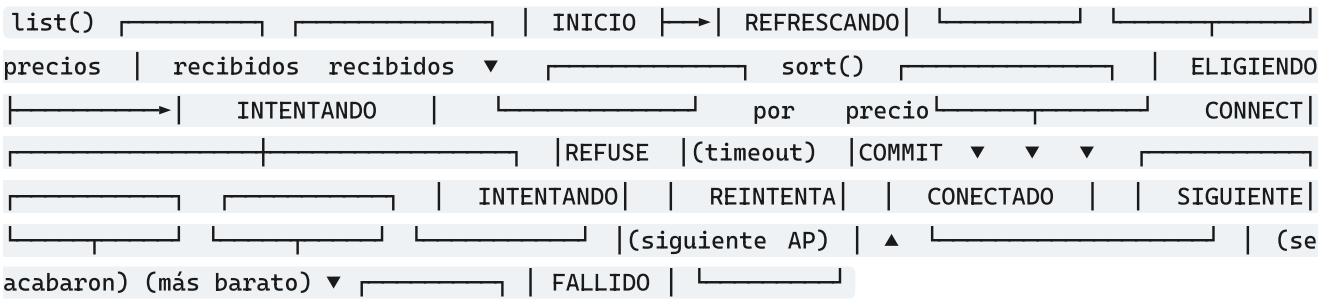
Observación 1 que suma puntos: la solución oficial no define ningún mensaje de **desconexión** ni de **error**. Un protocolo completo debería tener un (o el AP detecta la caída por timeout) y un para el caso de que el AP no responda. **Decí que el diseño oficial es mínimo y por qué eso es insuficiente en la práctica.**

Observación 2 — por qué es broadcast y los otros no:

- va por **broadcast** porque el usuario todavía **no sabe la IP del AP**: no la tiene, y es justamente el mensaje que se lo da. Si fuera unicast necesitaría conocer la IP destino. *(Y un broadcast requiere .)*
- **sí puede ser unicast**, porque en ese momento el usuario ya tiene la IP del AP (viene en el `OFFER`, o se la deduce del hecho de haberle recibido el).
- `COMMIT` y `REFUSE` van por unicast al usuario que mandó el `CONNECT`.

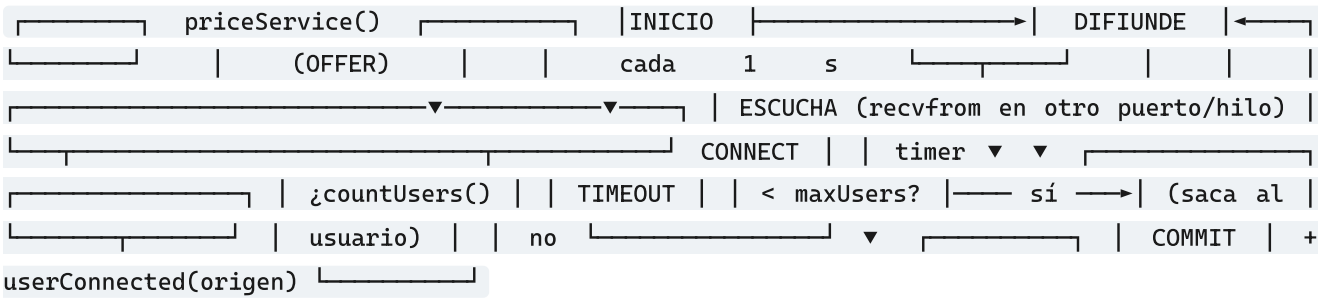
Parte 2 — Las máquinas de estado

Usuario:



Punto de atención: los tienen que distinguir "este AP no me sirve" de "no me pude comunicar". Un explícito avanza al siguiente AP de la lista ordenada; un **timeout** obliga a **reintentar el mismo** o a considerar que está caído. **La solución oficial no distingue los dos casos**, y esa es la diferencia entre un protocolo que funciona y uno que se cuelga.

AP:



Parte 3 — La implementación

La solución oficial, verbatim (con sus errores marcados):

`c void priceService(){ int socket = socket(AF_INET, SOCK_DGRAM, 0); while (true){ sleep(1); int broadcast = 1; setsockopt(socket, SOL_SOCKET, SO_BROADCAST, &broadcast, sizeof broadcast) // [E1]

struct sockaddr_in addr; socklen_t addr_size = sizeof addr; addr.sin_family = AF_INET; addr.sin_port = htons(12345); addr.sin_addr.s_addr = inet_addr(INADDR_BROADCAST); // [E2] char msg[5]; // [E3] sprintf(msg, %d, getPrice()); // [E4] int msg_size = strlen(msg); int sent_msg_size = sendto(socket, msg, msg_size, 0, (struct sockaddr*)&addr, addr_size); } } `

Los errores, uno por uno

#	Error	Consecuencia	Corrección
E1	**le falta el paréntesis de cierre**	No compila	
E2		** es una macro (un entero), no una cadena.** espera	
E3		Si el precio es 5 dígitos () **no entra**: desborda el buffer	o mejor
E4		**No compila**: faltan las comillas del formato	
E5	**adentro** del loop, junto al	El se re-setea 60 veces por minuto sin necesidad. Menor, pero es ruido	Sacar el **fuera** del loop
E6	se asigna y **nunca se usa**	El puede fallar () y el servicio no se entera	Verificar el retorno y decidir qué hacer
E7	No hay manejo de error en	Si falla, vale y todo lo demás falla en cascada	

El error E4 es el que más suma puntos, porque revela que **la solución oficial no fue probada**. Decirlo es el tipo de observación que el corrector valora.

La versión corregida de

`c

define PUERTO_PRECIO 12345

define MAX_PRECIO 16 / *alcanza para un int con signo + \0 /*

void priceService(void) { int fd, ssize_t enviados; int broadcast = 1; char msg[MAX_PRECIO];

fd = socket(AF_INET, SOCK_DGRAM, 0); if (fd < 0) { perror("socket"); return; }

/ *E5: esto se hace UNA vez, no en cada vuelta* / if (setsockopt(fd, SOL_SOCKET, SO_BROADCAST, &broadcast, sizeof broadcast) < 0) { perror("setsockopt"); close(fd); return; }

struct sockaddr_in destino; memset(&destino, 0, sizeof destino); destino.sin_family = AF_INET; destino.sin_port = htons(PUERTO_PRECIO); destino.sin_addr.s_addr = htonl(INADDR_BROADCAST); / *E2* /

for (;;) { sleep(1);

snprintf(msg, sizeof msg, "%d", getPrice()); / *E3 + E4* /

enviados = sendto(fd, msg, strlen(msg), 0, (struct sockaddr *)&destino, sizeof destino); if (enviados < 0) /* E6 / perror("sendto OFFER"); /* si es 0 o menor que strlen(msg), se perdio parte: hay que avisar */ } } `

Y corregida. Los errores de la oficial son más graves:

`c void connectCheaper(void) { int fd, i, r, n, n_ofertas = 0; int exito = 0; / *estado del protocolo* / char respuesta[MAX_PRECIO]; char **lista;

```
struct oferta {
    char *ssid;
    int   precio;
    struct sockaddr_in dir;             /* de donde vino el OFFER */
} *ofertas;

/* ================ 1. socket y bind al puerto 12345 ================ */
fd = socket(AF_INET, SOCK_DGRAM, 0);
if (fd < 0) { perror("socket"); return; }

struct sockaddr_in local;
memset(&local, 0, sizeof local);
local.sin_family      = AF_INET;
local.sin_port        = htons(PUERTO_PRECIO);
local.sin_addr.s_addr = htonl(INADDR_ANY);
if (bind(fd, (struct sockaddr *)&local, sizeof local) < 0) {
    perror("bind"); close(fd); return;
}

/* E-A: la oficial compara con "==" sobre punteros. Eso compara DIRECCIONES
   de memoria, no el contenido de las cadenas. SIEMPRE strcmp. */
/* E-B: ademas el literal va sin comillas, asi que ni siquiera compila. */
/* E-C: la oficial llama recv() en vez de recvfrom(), y no guarda de quien
   vino el OFFER: sin eso no sabe a que AP responderle el CONNECT. */
/* E-D: no hay timeout. Si el AP mas barato esta caido, recv() bloquea
   para siempre y el usuario se queda sin conexion. */
/* E-E: declara "int data_size" y lo usa sin inicializar. */

/* ================ 2. descubrir APs y recoger precios ================ */
lista = list();
n = length(lista);
ofertas = malloc(n * sizeof *ofertas);

/* E-D: sin timeout, un AP caido bloquea el bucle entero */
struct timeval tv = { .tv_sec = 2, .tv_usec = 0 };
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);

for (i = 0; i < n; i++) {
    struct sockaddr_in de;
    socklen_t le = sizeof de;

    associate(lista[i]);            /* el AP ya tiene nuestra IP por DHCP */

    /* E-C: recvfrom, no recv: necesito la direccion del emisor */
    r = recvfrom(fd, respuesta, sizeof respuesta - 1, 0,
                 (struct sockaddr *)&de, &le);
    if (r <= 0) continue;          /* nadie respondio: lo salto */

    respuesta[r] = '\0';

    /* E-F: la oficial no valida que el OFFER sea un numero */
    char *fin;
    long pr = strtol(respuesta, &fin, 10);
    if (fin == respuesta || *fin != '\0' || pr < 0) continue;

    ofertas[n_ofertas].ssid   = lista[i];
    ofertas[n_ofertas].precio = (int)pr;
    ofertas[n_ofertas].dir    = de;
    n_ofertas++;
}
```

```
if (n_ofertas == 0) {
    printf("no hay APs que respondan\n");
    free(ofertas); close(fd); return;
}

sort(ofertas);                      /* de menor a mayor precio */

/* ================ 3. intentar en orden de precio ================ */
for (i = 0; i < n_ofertas && !exito; i++) {
    char *req = "CONNECT";
    char resp[MAX_PRECIO];

    /* E-G: hay que asociarse ANTES de pedir la conexion; es lo que hace
       posible que el DHCP del AP exista cuando le llegue el CONNECT */
    associate(ofertas[i].ssid);

    sendto(fd, req, strlen(req), 0,
           (struct sockaddr *)&ofertas[i].dir, sizeof ofertas[i].dir);

    /* PROBLEMA DE DISEÑO que la oficial ignora: el socket esta bindeado al
       puerto 12345, pero el AP responde a NUESTRO PUERTO EFIMERO, no al
       12345. Este recv se queda esperando donde no va a llegar nada.

       Dos formas correctas de resolverlo:
         (a) hacer connect() sobre el socket UDP hacia cada AP: el SO fija
             el peer y las respuestas llegan por este mismo socket;
         (b) crear un socket efimero aparte para la fase de conexion. */
    r = recvfrom(fd, resp, sizeof resp - 1, 0, NULL, NULL);

    if (r > 0) {
        resp[r] = '\0';
        /* E-A corregido: strcmp, y el literal con comillas */
        if (strcmp(resp, "COMMIT") == 0) {
            connected();            /* la API confirma la conexion */
            exito = 1;
        }
        /* si es REFUSE, el for sigue con el siguiente AP mas barato */
    }
    /* si hubo timeout (r < 0), tambien seguimos con el i siguiente */
}

if (!exito) printf("No se pudo conectar a ningun AP disponible.\n");

free(ofertas);
close(fd);
```

}

Los 7 errores de la oficial, resumidos para el examen:

1. en vez de al comparar "COMMIT" (y el literal sin comillas).
2. en vez de : **no guarda la dirección del AP**, así que no puede dirigirle el `CONNECT`.
3. **No hay timeout**: si el AP más barato no responde, bloquea para siempre.
4. **sin inicializar**.
5. se llama en el bucle de precios pero **no se vuelve a llamar** antes del `OFFER`.
6. No valida que el OFFER recibido sea un número (sobre basura).

7. **El del AP vuelve al puerto efímero**, no al 12345 de los : la oficial no resuelve ese desajuste entre las dos fases.

> El punto 7 es el más fino y el que mejor diferencia a quien entendió el problema. La solución de un examen tiene que atacar **los dos lados** (AP y usuario), y si solo manda mensajes, el usuario nunca recibe las respuestas.

3.2 Examen 2009-07, Problema 2 (30 puntos) — cliente anti-spam vía HTTP

> **Fuente:** Examen / , julio de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0907.pdf

El enunciado describe una aplicación web de detección de spam: por cada correo que llega, se consulta y, si no es spam, se guarda.

Apartado a) — Implementar la consulta DNS por UDP

Enunciado: *"Implementar la invocación al DNS necesaria para obtener la IP del servidor de correo dado su nombre de dominio. Esto debe realizarse en una función en el cabezal: que devuelva la dirección IP en su formato decimal separado por puntos. Se debe realizar la invocación vía UDP, la misma que se utiliza para consultar cualquier otro host."*

Solución oficial (resumida del PDF):

```
c string getMailServer(string hostName) { int s = socket(AF_INET, SOCK_DGRAM, 0); struct sockaddr_in dest; /* armar la consulta DNS para hostName, tipo A */ enviar(s, hostName, (struct sockaddr *)&dest, sizeof dest); /* recibir la respuesta */ int r = recv(s, respuesta, sizeof respuesta, 0); /* ... buscar la primer entrada de tipo A ... */ return(ip_en_puntos, buf); }
```

Resolución completa. La estructura del pseudocódigo, con la cuenta de bytes que hay que hacer:

` PASO 1 — Socket UDP y destino

s = socket(AF_INET, SOCK_DGRAM, 0) destino.sin_family = AF_INET destino.sin_port = htons(53) ← DNS está en el puerto 53 destino.sin_addr = la IP del servidor DNS local (¡NO la del nombre que buscamos!exito)

PASO 2 — Armar la cabecera del mensaje DNS

El mensaje DNS empieza con 12 bytes de cabecera, en ORDEN DE RED: ID (2 bytes) = un número cualquiera, ej. 0x1234 flags (2 bytes) = 0x0100 → QR=0 (consulta), RD=1 (quiero recursión) QDCOUNT (2 bytes) = 0x0001 → una pregunta ANCOUNT, NSCOUNT, ARCOUNT = 0

Después va el nombre a consultar, en formato "etiqueta": 3www3fing4edu2uy0 (cada byte dice cuántos caracteres tiene la etiqueta; el 0 final marca el fin del nombre)

Después: QTYPE (2 bytes) = 0x0001 para tipo A QCLASS (2 bytes) = 0x0001 para clase IN

PASO 3 — Enviar y recibir

sendto(s, consulta, largo, 0, (struct sockaddr *)&destino, sizeof destino) r = recv(s, respuesta, MAX, 0) ← con recvfrom si querés saber qué servidor respondió

PASO 4 — Buscar la respuesta

En la respuesta, el ID DEBE coincidir con el que mandaste (si no, se descarta: puede ser una respuesta vieja de otra consulta). ANCOUNT dice cuántas respuestas hay. Con ANCOUNT = 0 y RCODE = 3 (NXDOMAIN) no existe ese nombre. La respuesta trae el mismo nombre, luego TYPE=1, CLASS=1, TTL (4 bytes) y finalmente los 4 bytes de la IPv4.

PASO 5 — Formatear

Convertir los 4 bytes a decimal separado por puntos con snprintf. Devolver ese string, o la cadena vacía si fallo. `

Errores frecuentes que se\columnwidth en este tipo de ejercicio:

- **Mandá ** en vez de : el enunciado dice explícitamente UDP.
- **Usar el puerto equivocado**: es el **53**, y hay que convertirlo con `htons`.
- **No verificar el ID**: la respuesta que llegue puede ser de una consulta anterior. El enunciado no lo dice, pero es la robustez mínima.
- **Devolver la primera IP del `Answer`** en vez de la del `Additional`.

Apartado c) — Dónde llamar a

Enunciado: modificar el pseudo del SMTP para invocar donde corresponda y guardar el correo solo si no es spam.

Resolución. El punto clave es **dónde**: la verificación va **después** de recibir el mensaje completo pero **antes** de escribirlo en el archivo.

` SMTP del cliente (pseudo de FING)

... else if (line.startsWith("DATA")) {

if (isSpam(getMailServer(hostRemitente), subject)) { / *no se guarda nada: se descarta* / rejection = "550 mensaje clasificado como spam"; } else { escribirCorreo(archivo, mensaje); OK = "250 mensaje recibido"; } ... } `

Las tres decisiones que hay que justificar:

1. **Después de `DATA`, no antes.** Antes no tenés el `Subject` ni el cuerpo, que es lo que necesita para clasificar.
2. **Antes de escribir en disco.** Si guardás y después descartás, el archivo ya quedó escrito: eso es precisamente lo que un antivirus bien hecho evita.
3. **El se llama sobre el remitente**, porque la API del servicio recibe la IP del servidor de correo del que vino el mensaje, no el nombre.

> **Tope para 30 puntos:** el 1er apartado vale más (es el que tiene la parte de programación), y el punto 3 del listado de errores es el que más suma: demuestra que pensaste en **por qué** la DNS se consulta y no solo en **cómo**.

PARTE 4 — CIERRE

📌 Resumen

1. Un **socket** es un par de descriptores de archivo más un estado que el SO mantiene: direcciones, buffers y (en TCP) la tabla de la conexión.
2. La API de sockets **no es parte de TCP/IP**: es del sistema operativo.
3. **Servidor TCP**: → → → → → /. ** devuelve un fd NUEVO.**
4. **Cliente TCP**: → → /. **Sin ni .**
5. **UDP**: , /, **no hay ni** , y es lo que te dice quién te mandó el datagrama.
6. / **en los dos lados**: la red es big-endian y el host little-endian. Sin la conversión, el puerto 80 llega como 20480.
7. es **obligatorio en el servidor** (si no, queda en un puerto efímero que nadie conoce) y **opcional en el cliente**.
8. **Sin** , un segundo al mismo puerto falla con .
9. La clave real de demux del kernel es la **5-tupla**; la "2-tupla de UDP" es la simplificación clásica. QUIC la reemplaza por el **Connection ID**.
10. **Wireshark** es la herramienta de laboratorio, y **HTTP/3 ya no se puede sniffear** con captura pasiva.

✅ Autoevaluación

1. ¿Por qué el servidor TCP hace pero el cliente no?

▶ Respuesta

2. Si escribís `dir.sin_port = 80;` sin `htons`, ¿qué puerto queda escuchando el servidor? ¿Y si la máquina fuera big-endian?

▶ Respuesta

3. ¿Qué devuelve y por qué el descriptor que devuelve no es el mismo que se le pasó?

▶ Respuesta

4. ¿Qué dos funciones de la API no existen para UDP, y por qué?

▶ Respuesta

5. ¿Por qué la solución oficial de usa para comparar con "COMMIT", y qué hay que usar?

▶ Respuesta

6. En el problema de `connectCheaper()`, ¿qué problema tiene el hecho de que los lleguen al puerto 12345 pero las respuestas (/) no regresen allí?

▶ Respuesta

🔗 Referencias

- **Examen / ** — Problema 1 completo (30 pts,), P2 (8 pts, mux/demux UDP)
- **Examen / ** — Problema 1 (30 pts, sockets y AX.25), Problema 2 (30 pts, anti-spam por HTTP)
- **Examen / ** — Problema 1: dos tarjetas de red en un mismo host
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6
- ** §14** — sockets, Wireshark, IPv6 dual-stack, 5-tupla vs. Connection ID
- **Clase siguiente:** 15 — DHCP, configuración y Simulacro 2